Defaults allow a generic type parameter to be optional by providing a fallback type if none is specified, such as `<T = string>`. Generics with constraints and defaults make TypeScript code more reusable and type-safe.

Choosing between generics and union types for function parameters balances flexibility, type safety, and maintainability. Generics support any type and maintain strong type relationships, making them ideal for components that must work with open-ended or unknown types and for functions where the output type depends on the input. Union types suit a fixed, known set of types —such as a parameter that can be a string or number—especially when no relationship between input and output is required or when simplicity is preferred. Union types may require type narrowing and can be less specific about return types, but are straightforward for simple cases. Generics are preferred for flexibility and strict type relationships; union types are best for limited, predictable sets of types and when simplicity outweighs reusability.

For this topic, are you able to answer these follow-up/probing questions?

- "What is the main advantage of using generics over the `any` type in TypeScript?"
- "How do generics improve code reusability in TypeScript?"
- "Explain the difference between a simple type parameter and a generic with constraints (`extends`)."
- "What is the role of the `keyof` operator in generic constraints, and how does it prevent runtime errors when accessing object properties?"
- "What are the trade-offs between using generics and using union types for function parameters, and when would you choose one over the other?"

3.9 Type safety and tools

TypeScript’s type system is more than just a bug prevention mechanism—it is a powerful foundation for building safe, maintainable, and large-scale applications. In technical interviews, assessment often focuses on the ability to apply type narrowing, assertions, and strict compiler configurations to ensure type safety across diverse and evolving codebases. Questions may probe not only familiarity with fundamental practices but also skill in